

Hashing: A technique in which a key is mapped to an address or a location in a data structure.

→ Better time complexity than linear or Binary search.

→ Not very good at space complexity.

* A good hash function should: (•) Minimise collisions. (•) Be easy and quick to compute, (•) Distribute key values evenly.

(1) Direct Hashing:

→ simple

→ Each key directly refers to address without any computation.

(2) Subtraction/Multiplication/Division/Addition:

$$\rightarrow h(k) = |k - x| \% N$$

N = size of table.

$$\rightarrow h(k) = |k + x| \% N$$

$$\rightarrow h(k) = (k * x) \% N$$

$$\rightarrow h(k) = (k // x) \% N$$

(3) Divide Modulo: (Remainder) → Prime Numbers give lesser no. of collisions.

$$h(k) = k \% N$$

→ Table size of power of 2 like 32, 1024 leads to more collisions.

→ powers of 10 for table size not good when keys rely on decimal integers.

* Prime numbers not close to powers of 2 are better table size values.

(4) Truncation/Digit Extraction:

→ Extract digits mentioned and combine

$$K = \underline{3445658}$$

$$h(k) = 3458 \% 1000 = \boxed{458}$$

→ Very fast but digit distribution in keys may not be even.

(5) Mid-square Method:

→ square the key → Extract all the middle digits.

$$k = 121, h(k) = 14641 = \boxed{464}$$

→ works well if keys do not contain a lot of trailing or leading zeros.

→ Non-integer keys have to be preprocessed to obtain corresponding integer values.

(6) Folding Method { Digit Folding / Fold shifting }:

→ Key is divided into n-parts of same digits given in question.

Example: $12345 \rightarrow 12 + 34 + 5 \rightarrow \boxed{51} \% N \Rightarrow \boxed{51}$

Fold Boundary :

→ Same as Fold shifting. $k \rightarrow k_1, k_2, k_3$.

→ Before adding, the first and last parts are reversed.

$k = 12\ 34\ 56\ 13\ 9789 \rightarrow k_1 = 123, k_2 = 456, k_3 = 139, k_4 = 789$

$$321 + 654 + 931 + 987$$

$$1903 \% 1000 = \boxed{903} \leftarrow \text{slot index.}$$

→ Folding method very useful if we have keys that are very large.

→ Fast and Simple.

(7) Rotation Method :

→ Used for keys assigned in a sequence; 1201, 1202, 1203, 1204.

→ Other methods end up with multiple collisions.

600101 $\rightarrow$ 160010

600102 $\rightarrow$ 260010

600103 $\rightarrow$ 360010

600104 $\rightarrow$ 460010

(8) Pseudorandom Generation :

→ Key is used to generate any pseudorandom number, then scaled up or down to fit address.

→ Used any expression $\{y = ax + c\}$ to generate a random number.

→ then $y \% N$.

$\downarrow$
 $a = \text{scaling co-efficient}$
 $c = \text{integer constant}$

(9) Radix conversion Method :

→ Transform a key into another number base.

e.g. → Map key 55354 to range 0 to 9999 using base 11. we have :

$$55354_{10} = 38652_{11}$$

Hashing Algorithm:

Algorithm hash (key, size, maxAddr, addr):

set loop to 0

set addr to 0

for each character in key

if (character not space)

add character to address

rotate addr 12 bits right.

end if.

end loop.

if (addr < 0):

addr = absolute(addr)

end if

addr = addr modulo maxAddr

end hash.

Collision Handling:

• Separate chaining:

→ requires additional data structure: {linked list}.

→ Multiple elements hashed into same slot index are inserted into a single-linked list forming a chain.

Advantages: → Simple to implement → used for unknown inserts/deletes.
→ Hash Table never fills up. → less sensitive to changes in hash function.

Disadvantages: → Not always const-time complexity → search time can become $O(n)$ in worst case if all key mapped into one slot index.
→ Wastage of space → Additional space for additional data structures.

Performance: Assumption under each key is likely to be hashed to any slot.

Load factor: $\lambda = \frac{k}{n} \times 100$ → k is no. of keys inserted
 n is total no. of slots/size.

→ if $\lambda \approx 1$, all keys getting inserted in one slot. collision ↑

→ $0.5 < \lambda < 0.7$ considered good choice

This algorithm converts an alphanumeric key of size characters into an integral address.
maxAddr is max possible address for the list.
addr contains hashed address.

Open Addressing: $\rightarrow$ No additional data structure, $\rightarrow$ Reshaping, $\rightarrow$ Useful when storage limited.
 $\rightarrow$ process of reshaping known as probing.

Types:

- 1) Linear Probing
- 2) Quadratic Probing
- 3) Double Hashing

Linear Probing
 If collision Resh key
 ① $h'(k) = [h(k) + 1] \bmod N$
 ② $h'(k) = [h(k) + 2] \bmod N$
 ③ $h'(k) = [h(k) + 3] \bmod N$
 ...

Advantages:

- $\rightarrow$ No additional data structures.
- $\rightarrow$ Data remains closer to home address.

Disadvantages:

- $\rightarrow$ Worst case $O(n)$ if searching becomes linear.

Quadratic Probing

$$h'(k) = [h(k) + 1^2] \% N$$

$$h'(k) = [h(k) + 2^2] \% N$$

$$h'(k) = [h(k) + 3^2] \% N$$

Advantages: Avoids clustering, No additional data structure.

Disadvantages:
 • More time required
 • May end up producing same slot index.

Double Hashing

- $\rightarrow$ Reduces clustering.
- $\rightarrow$ Uses another hash function.

- $\rightarrow$ May cause higher collisions due to randomness
- $\rightarrow$ May require ~~high~~ additional computation.

• Key off-set Reshaping:

$$h'(k) = [h_1(k) + (1 * h_2(k))] \bmod N$$

$$h'(k) = [h_1(k) + (2 * h_2(k))] \bmod N$$

$$h'(k) = [h_1(k) + (3 * h_2(k))] \bmod N$$

} $h_1(k)$ = primary hash function
 $h_2(k)$ = secondary hash function in case of collision.

Best Case: $O(1)$ $\rightarrow$ evenly distributed, all keys in every slot.

Worst Case: All keys in one slot.

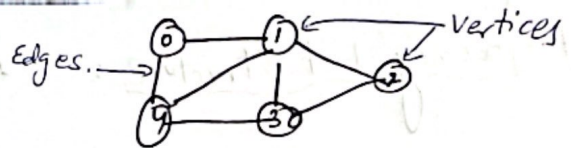
	List	Hash Table	
		Expected	Worst case
get item	$O(n)$	$O(1)$	$O(n)$
set item	$O(n)$	$O(1)$	$O(n)$
del item	$O(n)$	$O(1)$	$O(n)$
len	$O(1)$	$O(1)$	$O(1)$
iter	$O(n)$	$O(n)$	$O(n)$

DO
 WORKSHEETS!

and

Hashing (part 4) has some questions!

GRAPHS: $\{G(V, E)\}$



- V is vertices ~~also~~ which are nodes and E is edges which are lines/lines.
- Order of graph is no. of vertices in graph.
- Size of graph is no. of edges in graph.
- Endpoints: Two vertices joined by a single edge are called end vertices or endpoints.
- Adjacent vertices: Two vertices joined by a single edge.
- Incident edge: An edge is said to be incident to a vertex if the vertex is one of the edges endpoints.
- Outgoing edge: A vertex is said to have an outgoing edge when it has a directed edge whose origin is that vertex.
- Incoming edge: A vertex is said to have an incoming edge when it has a directed edge whose destination is that vertex.
- Degree of vertex: $\deg(v)$, no. of incident edges of a vertex.
- Indegree: no. of incoming edges of a vertex.
- out degree: no. of outgoing edges of a vertex.

Operation	Edge list	Adj-list	Adj-Map	Adj. Matrix
vertex-count()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
edge-count()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vertices()	$O(n)$	$O(n)$	$O(n)$	$O(n)$
edges()	$O(m)$	$O(m)$	$O(m)$	$O(m)$
get-edge(u, v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ exp.	$O(1)$
degree(v)	$O(m)$	$O(1)$	$O(1)$	$O(n)$
incident-edges(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
inside-vertex(x)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
remove-vertex(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
insert-edge(u, v, x)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$
remove-edge(e)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$

* m is no. of edges
 * Space complexity of Adj. Matrix $\rightarrow O(n^2)$
 * Space complexity of rest of data structures $O(n+m)$.

Edge List Structure: Edge list contains a tuple of addresses of connected vertices.

Space complexity: $O(n+m)$

Time complexity $\rightarrow$

Operation	Running Time
vertex-count(), edge-count()	$O(1)$
vertices()	$O(n)$
edges()	$O(m)$
get-edge(u, v), degree(v), incident-edges(v)	$O(m)$
insert-vertex(x), insert-edge(u, v, x), remove-edge(v)	$O(1)$
remove-vertex(v)	$O(m)$

Adjacency List Structure:

Adjacency Map Structure:

Adjacency Matrix Structure:

Graph Traversal:

→ Traversal is the procedure of exploring by examining all vertices and edges.

• Depth-First Search: { Uses Stacks }

Back Edge: which connects a vertex to its ancestor.

Forward edge: which connects a vertex to its descendant.

Cross edge: which connects two vertices that are neither ancestor nor descendant in the path

→ Directed Graph

* Time complexity: $O(n+m)$

• Breadth-First Search: { Uses Queues } { Same Time complexity }.

→ BFS is a good way to find shortest path from some starting vertex to every other vertex in a connected graph.

** Dijkstra's Algorithm: { Uses priority Queue }

Advantages: → Finds shortest path always ensuring optimal solution.

Disadvantages: → Does blind search so wastes a lot of time.
→ Cannot handle negative edges.
→ May lead to acyclic graphs.

Time complexity ↓ { V = no. of vertices, E = no. of edges } { n = no. of vertices, $n-1$ = no. of edges }
↓

Time for insertion into priority Queue = V

" " extracting minimum vertex = V^2

Time to update key values = E

Total Time = $V^2 + E = n^2 + n - 1$ | Hence, $O(n^2)$

TREES:

- T.root(): Returns the root node of the Tree, T or None if empty.
- T.is_root(): Returns True if node p is the root of tree T.
- T.parent(p): Return the parent of node p, or None if p is the root of T
- T.num_children(p): Return the no. of children of node p.
- T.children(p): Generate an iteration of the children of node p.
- T.is_leaf(p): Return True if node p does not have any children.
- len(T): Return the no. of nodes that are contained in tree T.
- T.is_empty(): Return True if tree T does not contain any nodes.
- iter(T): Generate an iteration of all nodes stored within tree T.

* Depth of root node is 0. → {Depth is no. of ancestors}

* Height of root node is 1. → {Height is no. of edges to its most distant leaf.}

↓
Empty Tree
w/ root node.

BINARY TREES:

* Root Node is assumed at Level 0.

* A binary tree is proper if each node has either zero or two children.

→ Balance Factor (B) is difference b/w height of its left subtree and right subtree

$$B = H_L - H_R$$

→ Left Heavy is $B > 0$, right heavy otherwise. If $B = 0$, its balanced.

Properties:

- 1) Max no. of Nodes at Level, L is 2^L
- 2) Max no. of nodes in binary tree of height, h is $2^h - 1$.
- 3) In a Binary Tree with N nodes, minimum possible height or minimum no. of levels is $\log_2(N+1)$.
- 4) Binary Tree with n_L leaves has at least $\text{ceil}(\log_2 n_L) + 1$ levels.
- 5) In a binary tree each node has 0 or 2 children, no. of leaf nodes is

$$n_L = n_i + 1$$

↑ ↑
leaf nodes internal nodes

- 6) If a binary tree has n_i internal nodes then total no. of nodes is;
 $N = 2n_i + 1$

7) And total no. of leaves is given by:

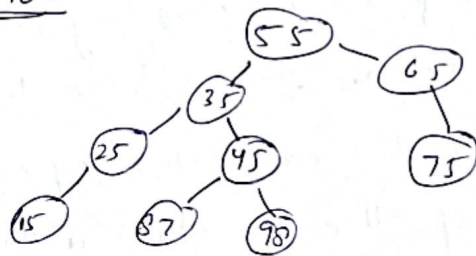
$$n_L = (N+1)/2 \Rightarrow N = 2n_L - 1$$

TREE TRAVERSAL:

1) Pre-order traversal:

- 1) Visit root node first and mark as visited.
- 2) Then traverse left sub-tree all way until leaf node.
- 3) Finally traverse right sub-tree until leaf node.

for example:



{ 55, 35, 25, 15, 45, 87, 98, 65, 75 }.

{ Root, Left, Right }.

2) Post-order traversal:

- 1) Start by traversing left-subtree of root until leaf node.
- 2) Then traverse right sub-tree until leaf node.
- 3) Finally, visit root node.

for example:

{ Same Tree as above }.

{ 15, 25, 87, 98, 45, 35, 75, 65, 55 }.

{ Left, Right, Root }.

3) In-order tree traversal:

→ specific to binary tree only.

- 1) start by traversing left sub-tree of root until leaf node.
- 2) Then, visit root node.
- 3) Finally traverse right sub-tree until leaf nodes.

for example:

{ Same Tree as above }.

{ 15, 25, 35, 87, 45, 98, 55, 65, 75 }.

{ Left, Root, Right }.

N = nodes

Worst-case for all traversals will be: $O(N)$

↑
prove through recurrence relation.

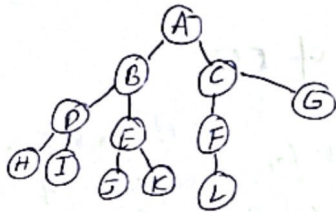
Array-based representation of Binary Tree:

- Position of left-child: $2f(p) + 1$
- Position of right-child: $2f(p) + 2$
- Position of parent: $\text{floor}[(f(p) - 1) / 2]$.

Linked-structure for Binary Tree:

{ root: root_node, Left: left_node, Right: right_node }.

for e.g.:



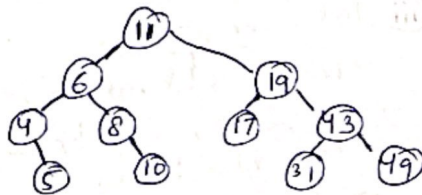
{ Root: A, Left: { Root: B, Left: { Root: D, Left: H, Right: I }, Right: { Root: E, Left: J, Right: K } }, Right: { Root: C, Left: { Root: F, Left: L, Right: None }, Right: G } }

Binary search Trees (BST):

→ An important application of inorder traversal algorithm.

How to construct trees:

{ 11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31 }
↑
Root Node.



① select first as root node.

② lesser than root goes to left

③ higher than root goes to right.

* For constructing a tree with height h , having n nodes, the time complexity will be: $O(n * h)$.

↳ multiply by h because it needs to search for the spot to insert as well, and it will take only one path and in worst case it goes all the way to the distant leaf node.

↳ hence, it iterates over all nodes (n).

① In worst case scenario, you can have just one single branch for all nodes being inserted for e.g. [10, 9, 8, 7, 6, 5, 4, 3, 2, 1].

↳ $O(n * n) \Rightarrow O(n^2)$.

② In best case scenario, tree would be perfectly levelled with max. height as $\log n$. for example: [5, 3, 2, 4, 7, 6, 8]

↳ $O(n * \log n)$

→ To find minimum keep going left from root node. $\Rightarrow O(h)$

→ To find maximum keep going right from root node. $\Rightarrow O(h)$

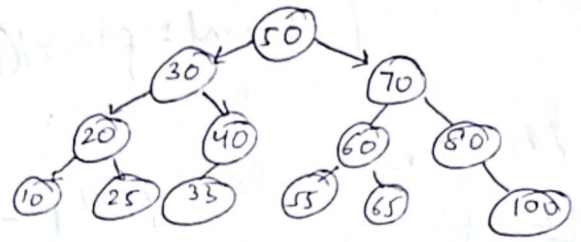
→ To find Successor: $\Rightarrow O(h)$

i) Successor of 50:

→ Right-child is a sub-tree

→ Successor is left-most child of its right sub-tree

→ 55.



ii) Successor of 80:

→ Right child is a single node

→ Right child becomes successor.

Start: 80 → Right: 100 → Left: Null

→ 100.

iii) Successor of 55:

→ No right child as it's a leaf node.

→ We look for ancestor with an immediately bigger value than 55.

→ Start: 55 → Parent: 60 → Stop

→ 60.

→ To find Predecessor: $\Rightarrow O(h)$

i) Predecessor of 50:

→ Left-child is a sub-tree

→ Predecessor is right-most child of its left subtree.

→ 40

ii) Predecessor of 40:

→ Left-child is a single node

→ Left-child becomes its predecessor.

→ 33.

iii) Predecessor of 65:

→ No left child, as 65 is a leaf node.

→ We look for ancestor with an immediate lesser value than 65.

→ Start: 65 → Parent: 60 → Stop.

→ 60.

Bubble Sort

→ Swap with neighboring element

→ In one pass it runs $(n-1)$ times

1st P

[A, S, A, D, L]

[A, S, A, D, L]

[A, A, S, D, L]

[A, A, D, S, L]

2nd P

→ Runs one extra pass times

→ Worst case (Reverse order list) $(n-1)$ passes.

Prefix → Infix (Stack)

• $XY^5Z^*/10+$

$\frac{X}{Y} (X^Y) \quad \frac{Z^*}{(X^Y)} (Z^*/(X^Y))$
 $\frac{Z^*}{X^Y} (Z^*/X^Y) \quad \downarrow$
 $[(Z^*/X^Y) + 10]$

Infix → Postfix (Stack)

• $2 * 54^2 + 20/4 + 44 - 3$
→ Use PEMDAS to solve.

$[(2 * (54^2) + (20/4) + 44) - 3]$

• $9^3 * 5^{15} * / 10 +$

$\{[(9^3)/(5 * 15)] + 10\}$

Infix → Postfix

$(A + (B * C)) - D$

$\begin{array}{|c|} \hline C \\ \hline A \\ \hline \end{array} \quad \begin{array}{|c|} \hline * \\ \hline A B C * + \\ \hline \end{array}$
* After + comes closing bracket so it pops until open bracket
 $\begin{array}{|c|} \hline C \\ \hline A B \\ \hline \end{array} \quad \begin{array}{|c|} \hline - \\ \hline A B C * + D - \\ \hline \end{array}$

Quick Sort

→ partition function:

$i = L$

$j = H$

while ($i < j$):

while ($arr[i] \leq pivot$):

$i = i + 1$

while ($arr[j] > pivot$):

$j = j - 1$

if ($i < j$):

swap $arr[i]$ with $arr[j]$

$P-index = \text{get index of pivot}$

swap $arr[i]$ with $arr[P-index]$

return j

→ Time efficiency depends on pivot of choice.

→ Faster than Merge if right pivot.

→ Best: $O(n \log n)$

Worst: $O(n^2)$

Avg: $O(n \log n)$

Formulas:

→ Sum of first n natural numbers

$$\Rightarrow \frac{n(n+1)}{2}$$

→ Sum of powers of 2.

$$2^0 + 2^1 + \dots + 2^k$$

$$\Rightarrow 2^{k+1} - 1$$

→ Sum of squares

$$1^2 + 2^2 + \dots + n^2$$

$$\Rightarrow \frac{n(n+1)(2n+1)}{6}$$

→ Sum of odd series

$$1 + 3 + 5 + \dots + (2n-1)$$

$$\Rightarrow n^2$$

List ADT:

insert-first():

insert-last():

remove-first():

remove-last():

get-first():

get-last():

is-full():

is-empty():

Linear Search Algorithm:

$\begin{array}{|c|} \hline 15 \\ \hline \end{array} \quad \begin{array}{|c|c|c|c|} \hline 13 & 48 & 7 & 15 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 15 \\ \hline \end{array} \quad \begin{array}{|c|c|c|c|} \hline 13 & 48 & 7 & 15 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 15 \\ \hline \end{array} \quad \begin{array}{|c|c|c|c|} \hline 13 & 48 & 7 & 15 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 15 \\ \hline \end{array} \quad \begin{array}{|c|c|c|c|} \hline 13 & 48 & 7 & 15 \\ \hline \end{array}$

→ Can be applied to both sorted and unsorted data.

Binary Search:

→ Must be sorted

→ Search starts from middle element.

→ Compare middle value

→ eliminates and updates value of High & Low.

③ $\begin{array}{|c|c|c|c|c|} \hline L & M & H & & \\ \hline 2 & 3 & 7 & 11 & 24 \\ \hline \end{array}$
 $\begin{array}{|c|c|} \hline L & H \\ \hline 2 & 3 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline M \\ \hline 3 \\ \hline \end{array}$
0 $\begin{array}{|c|} \hline 3 \\ \hline \end{array}$